

Predicted Futures Are Not Enough: Learning Executable Goals for Robot Manipulation

Tzu-Yu Chuang*, Ching-Hsiang Chang[†], Yi-Hsiu Lee[†], Yi-Ting Chen*, Min Sun[†], YuanFu Yang*

*National Yang Ming Chiao Tung University

[†]National Tsing Hua University

Abstract—Generative world models provide rich predictions of how manipulation scenes may evolve toward task objectives, yet those futures do not directly expose the compact task variables required by control. When training supervises future prediction alone, terminal goal accuracy is not an explicit learning objective, even when geometric recovery is available. We present *Entity-Level Goal Readout*, a learned prediction-to-execution interface that makes the executable terminal goal an explicit output of a 3D trace world model. It combines object-centric pose prediction with translation grounded in observed depth to produce a compact goal in $SE(3)$. A shared Pose-Native Executor consumes this fixed goal with online object-pose feedback for closed-loop control without rerunning the world model. Across five manipulation tasks, the pipeline achieves a mean success rate of 79.69%. Goal diagnostics directly measure terminal goal accuracy, while controlled translation perturbations characterize how execution degrades under goal error. Zero-shot deployment on a Franka arm achieves 73.33% success on nominal StackCube, 66.67% with distractors, and 75.00% on PickPlate with a target unseen during policy training. These results support treating the prediction-to-execution interface as an explicit learned component of world-model planning rather than incidental post-processing in the control pipeline itself. Project page: <https://claire0730.github.io/executable-goals/>

I. INTRODUCTION

Generative world models support robot manipulation by predicting how scenes can evolve toward task objectives. Yet rich future predictions do not by themselves expose the compact task variables required by control. Video, point trajectories, and scene geometry require a prediction-to-execution interface. Figure 1 frames this gap between prediction and execution. Geometric fitting can convert predicted motion into a terminal goal, but future prediction training does not optimize recovered goal accuracy. Prediction errors can still produce inaccurate control targets. Figure 2 illustrates inconsistencies in generated futures. Our comparisons show that terminal goal errors remain after predictor training losses converge. Converged prediction objectives do not guarantee accurate goals for control.

The design question is not whether a conversion procedure exists, but whether the task variables consumed by control are explicitly learned and exposed. We make terminal goal prediction part of the learning objective through object-centric prediction and observed geometry. The prediction-to-execution interface is learned as part of the model rather than left to incidental post-processing. We present *Entity-Level Goal Readout*, a learned interface between a 3D trace

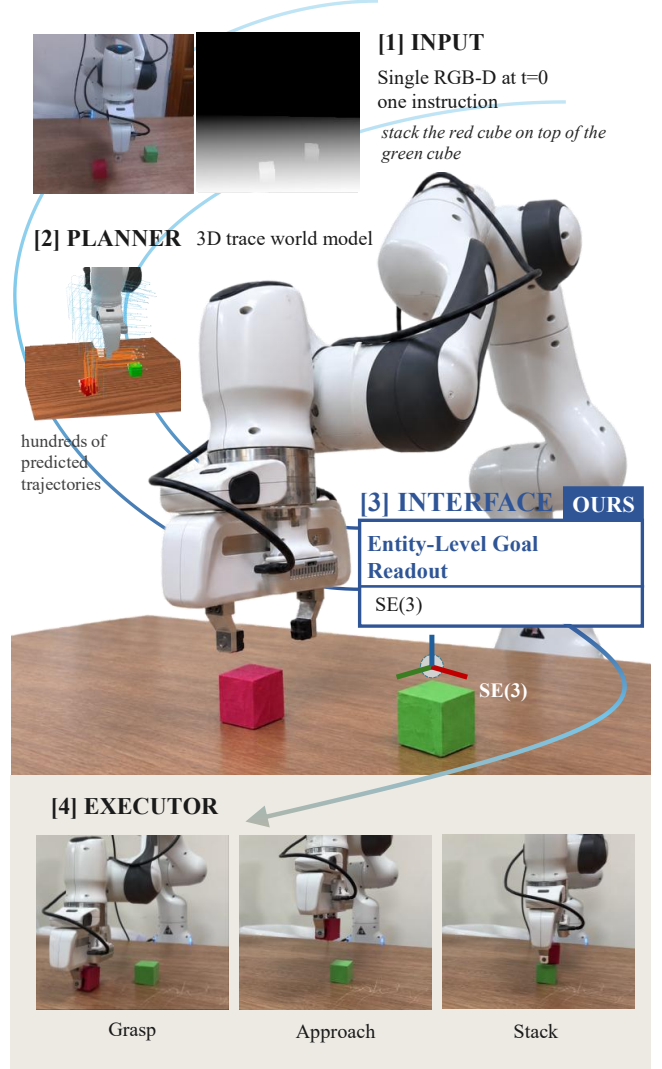
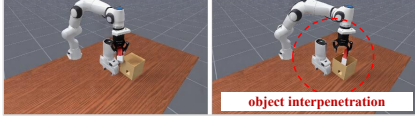


Fig. 1. **From prediction to an executable goal.** A single RGB-D observation and task instruction condition a 3D trace world model. The Entity-Level Goal Readout converts the resulting world-model representation into a compact object-relative goal in $SE(3)$ for the Pose-Native Executor. The StackCube example illustrates the prediction-to-execution interface from a rich predicted future to closed-loop manipulation.

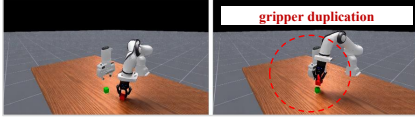
world model and a Pose-Native Executor. Figure 3 traces this interface from the world-model representation to closed-loop execution. An entity branch predicts terminal object pose,

[A] VIDEO WORLD MODELS · PIXELS ONLY

Cosmos 3 Edge

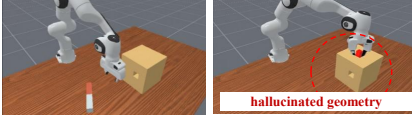


PegInsertionSide · insert the peg into the hole in the box

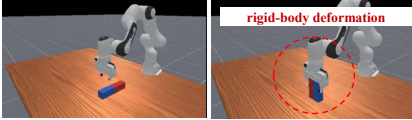


StackCube · stack the red cube on top of the green cube

Veo 3.1



PegInsertionSide · insert the peg into the hole in the box



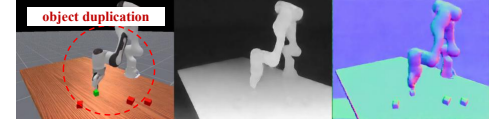
LiftPegUpright · lift the peg and stand it upright

[B] 4D WORLD MODELS · PIXELS + GEOMETRY

Tesseract - RGB | depth | normal



cross-channel inconsistency



StackCube · stack the red cube on top of the green cube

Fig. 2. **Qualitative motivation.** Cosmos 3 Edge [1], Veo 3.1 [2], and Tesseract [3] illustrate interpenetration, hallucinated geometry, duplicated objects or grippers, deformation, and inconsistent RGB, depth, and normals. These prediction defects motivate evaluating the control goals recovered from predicted futures. The separate goal diagnostic in Table I excludes Veo 3.1.

while a map branch grounds translation in observed depth to construct a compact executable goal in $SE(3)$. A shared executor combines this goal with pose feedback to correct execution error without rerunning the world model. This separates episodic planning from continuous control while preserving the task variables required by control.

We evaluate the interface through goal accuracy, manipulation success, and sensitivity to goal error. Across five tasks, our pipeline achieves higher mean success than both rigid readout pipelines under shared executor weights, with the largest gains on PickCube and StackCube. Goal error distributions characterize the targets delivered by the planner, while controlled translation perturbations measure the fixed executor’s response to imposed goal error. Real robot trials extend evaluation to distractors and a target unseen during policy training. Our contributions are threefold:

A Learned Goal Interface. Entity-Level Goal Readout makes control-relevant terminal variables explicit learning targets by combining object-centric pose prediction with depth-grounded spatial readout to expose executable goals from the world-model representation.

Execution Trained for Imperfect Goals. A shared Pose-Native Executor is trained under measured planner goal errors using privileged teachers and online pose feedback, without rerunning the planner or receiving privileged guidance during execution.

An Empirical Account of Goal Usability. Goal diagnostics across prediction representations, shared-executor evaluations, controlled perturbations, and real robot trials provide an empirical account of how goal quality translates into closed-loop execution.

II. RELATED WORK

Generative World Models. World model planners represent future motion as generated observations, point trajectories, or scene geometry. Dream2Flow [4] and Tesseract [3] recover motion or object goals from generated observations. ATM [5] and Track2Act [6] predict 2D point trajectories, with Track2Act recovering geometric poses for control. TraceGen [7] and μ_0 [8] predict 3D traces, PointWorld [9] predicts action-conditioned 3D motion, and Structured-4D [10]

forecasts scene geometry. Across these representations, our focus is how predicted futures become task goals consumed by control.

From Predicted Futures to Executable Goals. Geometric readouts recover control targets through pose estimation or rigid registration, including Kabsch fitting with random sample consensus (RANSAC) [11], [12]. Dream2Flow [4] connects video generation to control through 3D object flow. Such interfaces determine how predicted futures are exposed to control, but future prediction supervision alone does not directly optimize terminal goal accuracy. Our readout makes the terminal goal an explicit learning target and evaluates the resulting goal quality in Table I.

Goal-Conditioned Control. Diffusion Policy [13], DP3 [14], and vision language action models map rich observations to actions. Our Pose-Native Executor instead receives one fixed pose goal in $SE(3)$ per episode. Dataset aggregation and privileged teacher methods motivate its distillation strategy [15], [16], [17]. This keeps the prediction-to-execution handoff compact while separating goal quality from closed-loop execution.

III. METHOD

A. Overview and Problem Statement

The Entity-Level Goal Readout provides a learned prediction-to-execution interface between a 3D trace world model and goal-conditioned control, as shown in Fig. 3. The system separates planning once per episode from closed-loop execution. Given a single RGB-D observation and task instruction, the world model predicts future scene motion. From the shared world model representation, the interface explicitly predicts one executable goal $\Delta\hat{T}_g \in SE(3)$ relative to the initial object frame.

Execution runs independently at 20 Hz. An online perception module estimates the current object pose $\hat{T}_o(t)$ in the world frame. A compact Pose-Native Executor maps the current pose, fixed executable goal, robot proprioception, and previous action to commands for seven arm joint increments and a gripper position target. The planner communicates a compact seven-parameter representation of the $SE(3)$ goal

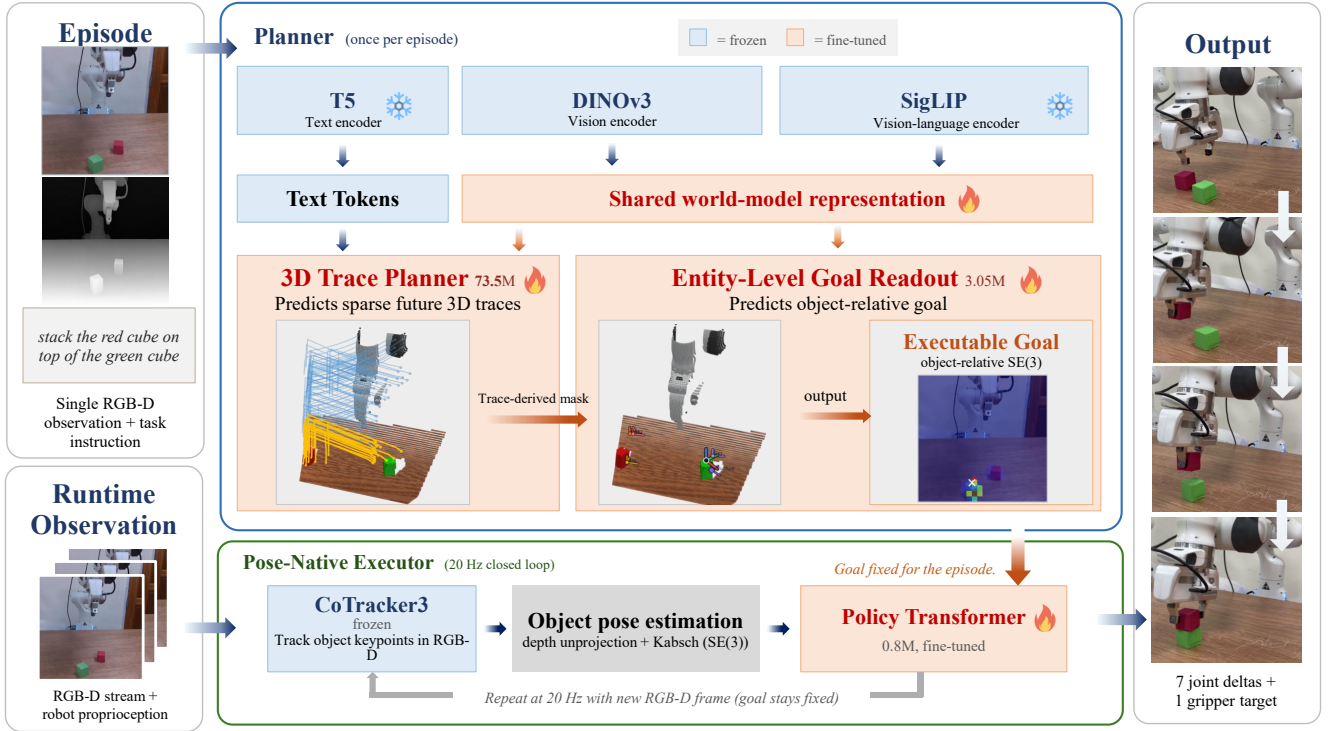


Fig. 3. **System overview.** Frozen encoders supply a shared world-model representation for trace prediction and Entity-Level Goal Readout. The readout exposes one object-relative goal, anchored at $t = 0$ and fixed for the episode. Online tracking and pose estimation update object state to close the loop. The Pose-Native Executor combines pose, goal, proprioception, and previous action to produce an eight-dimensional arm and gripper command at 20 Hz.

rather than the full predicted future. The planner and rigid baseline are described in Sec. III-B, the readout in Sec. III-C, and the executor and its training in Sec. III-D. Task success is evaluated separately from goal accuracy, as described in Sec. IV.

B. Trace Planner and Rigid Readout

Backbone. We build on TraceGen [7], which predicts sparse 3D point trajectories rather than rendered future frames. DINOv3 [18] and SigLIP [19] features are arranged on a 24×24 spatial grid and augmented with depth, while a frozen T5 [20] encoder embeds the task instruction. A flow matching decoder based on CogVideoX [21] predicts trajectories for 400 query points over 32 future steps.

The visual and language encoders remain frozen while planner layers adapted to each task are fine tuned on manipulation demonstrations. Evaluation uses the checkpoints and readout routes specified in Sec. IV-B.

Rigid Readout. The conventional *rigid readout* recovers a terminal transform from predicted object tracks using weighted Kabsch [11] inside RANSAC [12]. This provides a strong geometric baseline, but the underlying predictor is not directly trained for terminal goal accuracy, and the task goal must be recoverable as a single rigid transform. Here K denotes the number of stochastic trace predictions. For $K = 4$, traces are averaged before rigid fitting rather than selecting or averaging solved poses.

C. Entity-Level Goal Readout

Figure 4 details the two branches that explicitly predict task relevant quantities from the shared planner representation.

Entity Branch. An object mask derived from motion selects the relevant support on the 24×24 vision token grid. A learned query performs masked attention pooling to produce a single entity token. Cross attention to the 576 scene tokens enriched with geometry yields an object representation from which we predict the terminal object pose, whose rotation is used for executable goal composition. Keyframe poses, contact point, and contact normal are predicted only as auxiliary supervision and are not passed to the executor.

Map Branch. Terminal position is grounded directly in observed scene geometry. From the same spatial geometry tokens, a linear head predicts logits over a 24×24 goal map. After softmax and upsampling, the distribution is masked by valid observed depth and reduced to an image centroid weighted by depth. Unprojection through the RGB-D camera model yields a camera frame point. The calibrated camera to world transform T_{WC} maps it to the world position $\hat{\mathbf{t}}_g$. This readout also supports targets defined by regions that do not correspond to a unique rigid transform.

Goal Composition. Eq. (1) combines the map position with the entity rotation, both expressed in the world frame, to form the terminal object pose.

$$\hat{T}_g = \begin{bmatrix} \hat{R}_g & \hat{\mathbf{t}}_g \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3), \quad (1)$$

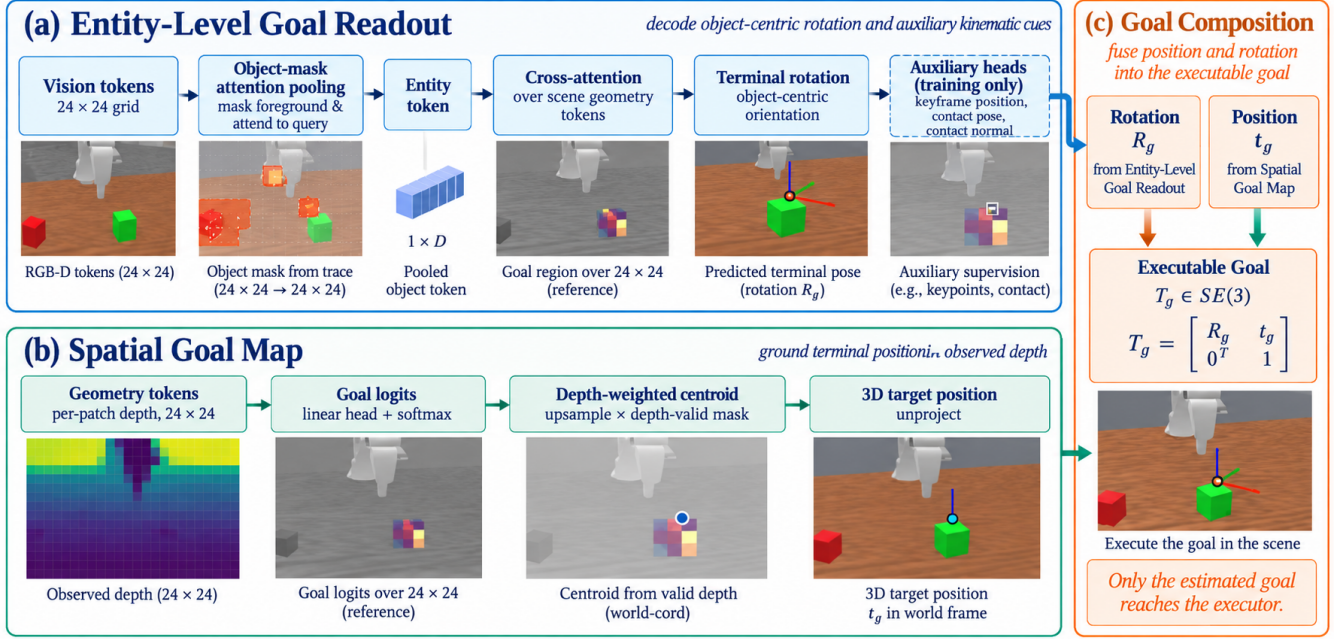


Fig. 4. **Entity-Level Goal Readout.** (a) Masked pooling and cross attention produce terminal rotation, with dashed paths for training-only auxiliary supervision. (b) The map branch grounds position in valid observed depth. (c) Goal composition combines rotation and position into the executable goal. Image panels illustrate intermediate representations.

The interface expresses this pose relative to the initial object frame as $\Delta \hat{T}_g = \hat{T}_o(0)^{-1} \hat{T}_g$. Here $\hat{T}_o(0)$ maps initial object coordinates to world coordinates.

Training Objective. The readout is trained jointly with the trace objective in Eq. (2).

$$\mathcal{L} = \mathcal{L}_{\text{flow}} + \lambda_{\text{ent}} \mathcal{L}_{\text{ent}} + \lambda_{\text{map}} \mathcal{L}_{\text{map}}, \quad (2)$$

where $\mathcal{L}_{\text{ent}}$ supervises terminal pose and auxiliary interaction quantities and $\mathcal{L}_{\text{map}}$ supervises the spatial goal distribution. These terms directly supervise executable goal prediction rather than relying on goals recovered from future traces after prediction.

D. Executor and Distillation

The readout runs once at episode start. The executor reconstructs the fixed world goal as $\hat{T}_g = \hat{T}_o(0) \Delta \hat{T}_g$ and compares it with the online world pose $\hat{T}_o(t)$. Both use the same world frame. The goal remains fixed while pose feedback updates the current state, correcting execution without rerunning the world model.

Online Perception. An initial object mask supplies tracking points in RGB-D observations. Frozen CoTracker3 [22] maintains their image correspondences. Depth unprojection followed by Kabsch recovers $\hat{T}_o(t)$, which is updated while the executable goal remains fixed.

Policy. The compact 0.8M-parameter transformer receives the current object pose, executable goal, joint positions and velocities, tool center point pose, and previous action. Its eight dimensional output comprises seven arm joint increments and a gripper position target at 20 Hz. The same student policy is used across all five tasks. Current pose feedback supports

correction during execution, while the fixed goal decouples control updates from planner inference.

Distillation. We use dataset aggregation (DAgger) [15]. The student controls the environment, and the privileged task policy supplies action labels at the visited states. The teacher additionally receives a privileged guidance variable ψ , which is not an input to the student during training or deployment. Goal perturbations are sampled from the measured planner error distribution during student training, exposing the controller to imperfect goals rather than only designated correct goals. Privileged guidance therefore enters through teacher supervision, not through the student’s observation at execution.

IV. EXPERIMENTS

We evaluate the prediction-to-execution interface through terminal goal quality across prediction representations, closed-loop success under a shared executor, and controller sensitivity to imposed goal error. Real robot trials extend the evaluation beyond simulation.

Evaluation Protocol. We use five ManiSkill3 [23] tasks. Position comparisons focus on PickCube, PegInsertionSide, and StackCube. LiftPegUpright admits multiple upright positions and PushCube accepts a broad target region, so both remain in success evaluations. Archived rigid and Oracle results pool three evaluation seeds with 256 scenes each, yielding 768 episodes per task and goal source. Pose diagnostics use 256 matched scenes. Perturbations use two evaluation seeds and 512 episodes per point. These protocols are separate from policy training. Simulation training and timing use a single RTX 5090.

TABLE I
TERMINAL GOAL POSITION ERROR IN MM, REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER VALID READOUTS. SOURCE-SPECIFIC STANDARD DEVIATION CONVENTIONS ARE RETAINED AND DOCUMENTED IN THE SUPPLEMENTARY APPENDIX. LOWER IS BETTER. EVALUATION BANKS AND VALIDITY CONVENTIONS ARE SPECIFIED IN SEC. IV.

Prediction	Method	PickCube	PegInsertionSide	StackCube
RGB video	Cosmos 3 Edge	141.3 $\pm$ 89.3	348.2 $\pm$ 214.9	87.6 $\pm$ 61.6
RGB-DN video	Tesseract, adapted	456.9 $\pm$ 158.3	76.9 $\pm$ 38.2	224.8 $\pm$ 89.0
2D tracks	Track2Act, adapted	167.5 $\pm$ 57.0	256.2 $\pm$ 33.0	143.0 $\pm$ 89.0
4D geometry	Structured-4D, released	192.2 $\pm$ 82.9	373.5 $\pm$ 115.3	31.0 $\pm$ 19.3
4D geometry	Structured-4D, adapted	166.4 $\pm$ 61.3	361.4 $\pm$ 83.3	36.3 $\pm$ 20.3
3D tracks	TraceGen + Rigid Readout ($K = 1$)	41.1 $\pm$ 21.7	42.6 $\pm$ 31.6	28.9 $\pm$ 38.7
3D tracks	TraceGen + Rigid Readout ($K = 4$)	37.2 $\pm$ 19.3	40.8 $\pm$ 33.0	25.4 $\pm$ 38.8
3D tracks + goal	Entity-Level Goal Readout	31.5$\pm$20.1	29.4$\pm$16.4	12.8$\pm$34.0

Table I measures 3D Euclidean distance between recovered terminal goal positions and reference goal positions, not image prediction or scene reconstruction error. It uses the Structured-4D evaluation bank, the diagnostic bank for the other external planners including adapted Track2Act, and stored goals for the final three rows. Statistics include finite estimates marked unsolved. The table and discussion use all valid readouts for each model rather than a common valid subset across checkpoints.

Baselines. Table I groups planners by RGB or RGB-DN video, 2D or 3D point trajectories, and 3D scene evolution. The final three rows use the same 3D point trajectory representation and compare rigid recovery with learned goal readout, focusing the comparison on how predicted futures are exposed to control. Table II separates native executors from the shared executor comparison. Oracle Goal rollouts use designated correct goals.

A. Goal Quality Across Planners

Fine Tuning. All three baseline adaptations run on a single RTX 5090 across five tasks. Tesseract generates RGB-DN video and uses low rank adaptation (LoRA) on 4,962 clips. Structured-4D predicts evolving 3D scene geometry with a two-stage generator adapted using LoRA on 9,120 frame pairs. Track2Act predicts 2D point trajectories and uses full model fine tuning on 4,493 clips. Training losses converge, but recovered terminal goals retain substantial errors. Training data and optimization budgets differ across methods.

Structured-4D reconstructs geometry at roughly centimeter scale, yet substantial PickCube and PegInsertionSide goal errors remain. Adapted Tesseract has lower position error on PegInsertionSide than on PickCube or StackCube, but substantial errors remain. Convergence under the prediction objective does not ensure accurate terminal goals. Within the shared 3D trace representation, Entity-Level Goal Readout gives the lowest mean position error on all three reported tasks. Its reported errors are 15.3%, 27.9%, and 49.6% lower than rigid $K = 4$ on PickCube, PegInsertionSide, and StackCube, respectively (Table I).

B. Closed-Loop Execution

Comparison Scope. Shared-executor comparisons use different checkpoints and samples, one policy-training seed, and noise matched to final-pipeline errors. Rigid readout uses the earlier trace checkpoint for four tasks. Final routing uses the goal-map checkpoint for PickCube and StackCube and position-head banks elsewhere. This supports pipeline comparison, not isolated readout attribution. Native executors provide separate system-level context.

Under shared executor weights, the final pipeline achieves 79.69% mean success, compared with 54.45% for $K = 1$ and 59.87% for $K = 4$ (Table II). Its mean success is 19.82 points higher than $K = 4$. The final pipeline exceeds both rigid variants on four tasks, with the largest gains over $K = 4$ on PickCube and StackCube. PushCube remains near saturation at 99.20%, slightly below both rigid variants. Additional trace averaging improves rigid readout on most tasks but does not close the mean success gap.

Adapted Structured-4D achieves higher StackCube success (89.00% versus 86.54%, Table II), but uses weights trained with 1,000 demonstrations per task and its native executor. Its 1.1B planner takes 4.45 s per plan versus 1.27 s for our 677.6M planner (Table III), while our executor is shared across five tasks.

Oracle Reference. Gray segments in Fig. 5(a) indicate execution difficulty even with designated correct goals. The updated LiftPegUpright result exceeds its archived Oracle reference, so these measurements remain empirical reference points rather than theoretical upper bounds.

C. Readout Diagnostics

The final pipeline reduces both typical and tail position error on StackCube in Fig. 5(b) and Fig. 6, while PegInsertionSide shows a clearer tail reduction. Quantiles exclude nonfinite estimates, but execution denominators retain all scenes. The 25 mm StackCube proxy improves even when finite unsolved goals count as failures, whereas PegInsertionSide’s 20 mm proxy changes little, showing that a single threshold does not fully characterize goal quality.

Raw quaternion errors in Fig. 5(c) are lower for the final PegInsertionSide bank, which uses the entity-branch

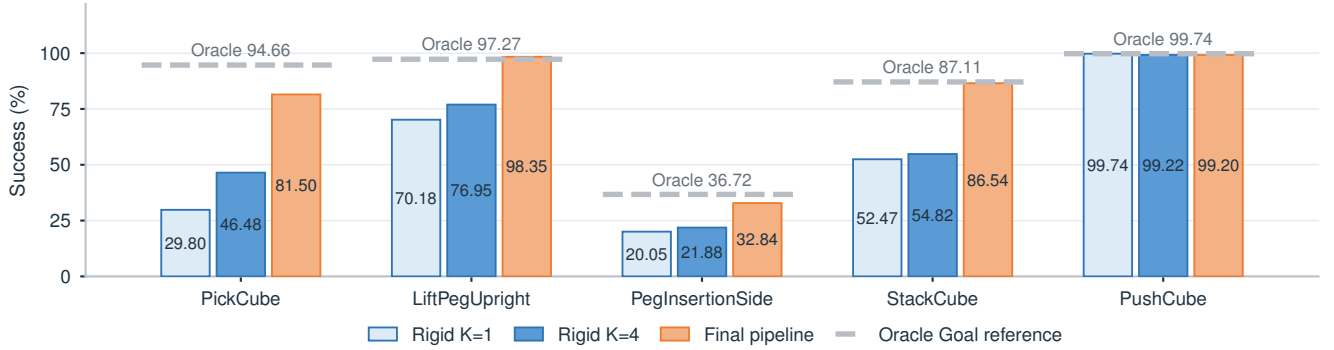
TABLE II

CLOSED-LOOP TASK SUCCESS. EXTERNAL SYSTEMS USE NATIVE EXECUTORS. THE SHARED-EXECUTOR BLOCK USES THE SAME POLICY WEIGHTS AND PAIRED SCENES, BUT DIFFERENT GOAL PIPELINES AND PLANNER CHECKPOINTS. NATIVE-EXECUTOR RESULTS ARE SEPARATE SYSTEM-LEVEL EVALUATIONS. RATES ARE PERCENTAGES. MEAN IS THE UNWEIGHTED AVERAGE OF ALL FIVE DISPLAYED TASK RATES, ROUNDED TO TWO DECIMAL PLACES. IT IS OMITTED FOR INCOMPLETE ROWS. BOLD AND UNDERLINE MARK THE BEST AND SECOND-BEST RESULTS IN EACH COLUMN ACROSS BOTH BLOCKS, RANKED USING THE REPORTED VALUES. DASHES DENOTE UNREPORTED RESULTS. ORACLE GOAL REFERENCES ARE SHOWN SEPARATELY IN FIG. 5(A).

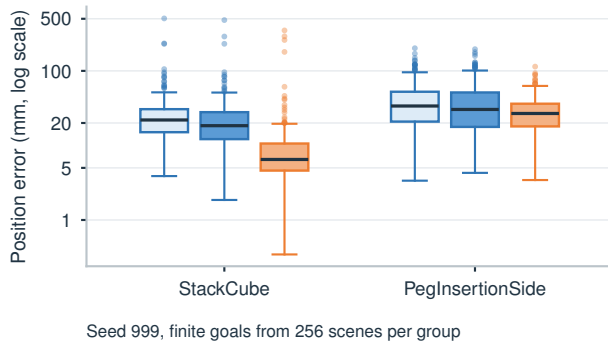
Method	Success (%)					Mean
	PickCube	LiftPegUpright	PegInsertionSide	StackCube	PushCube	
<i>Native executor</i>						
DP3 [14]	36.00	<u>84.00</u>	12.00	20.00	93.00	49.00
RoboFlow4D [24]	14.00	–	–	12.00	45.00	–
Structured-4D, released	0.00	0.00	16.00	84.00	0.00	20.00
Structured-4D, adapted	33.00	16.00	20.00	89.00	92.00	50.00
Cosmos 3 Edge Policy, adapted	3.00	0.00	0.00	13.33	23.33	7.93
<i>Shared executor</i>						
Rigid Readout ($K = 1$)	29.80	70.18	20.05	52.47	99.74	54.45
Rigid Readout ($K = 4$)	<u>46.48</u>	76.95	<u>21.88</u>	54.82	<u>99.22</u>	<u>59.87</u>
Entity-Level Goal Readout	81.50	98.35	32.84	<u>86.54</u>	99.20	79.69

(a) Success across five tasks with $K=1$, $K=4$, and final pipeline

Rigid readout and final pipeline with Oracle Goal references



(b) Position error distributions



(c) Stored goal rotation distributions

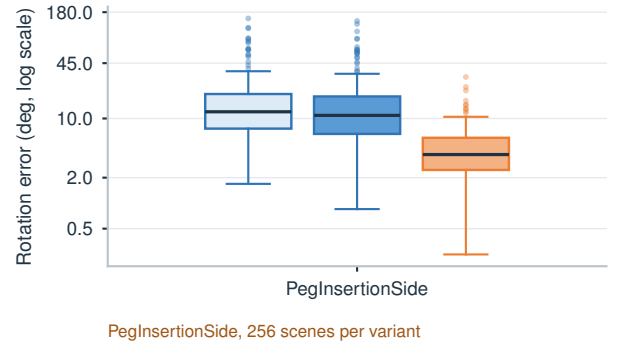


Fig. 5. **Task success and goal error distributions.** (a) Light blue, blue, and orange denote rigid $K = 1$, rigid $K = 4$, and the final pipeline. Gray segments show empirical Oracle Goal references. (b,c) Seed 999 box plots show the median and interquartile range (IQR), 1.5 IQR whiskers, and all outliers on logarithmic axes. Groups have 256 finite records except StackCube rigid positions (252/256). (c) Raw quaternion errors are shown for PegInsertionSide. Comparison conditions are in Sec. IV-B.

pose prediction. Removing geometry from the entity-position head increases error in Fig. 6, indicating reliance on scene geometry in this probe rather than a full training-loss ablation. Quaternion errors are not symmetry adjusted and do not establish rotation-mediated execution gains. Final

and $K = 4$ LiftPegUpright rotations are identical, while its upright predicate and StackCube symmetry require task-aware metrics, so no learned-rotation gain is claimed for either task.



Fig. 6. **Position Quality and Geometry Probe.** Median / 90th-percentile errors in mm. Lower is better. Final denotes the routed pipeline. No geo. removes geometry from the entity-position predictor at test time. It is not an ablation of Final. Segments connect quantiles, not confidence bounds. All groups use 256 scenes, with 252 finite StackCube rigid positions and 256 otherwise.

Position perturbation (512 episodes per point)

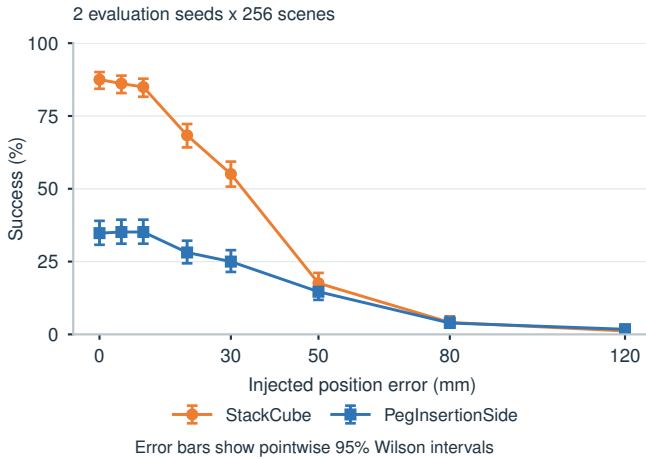


Fig. 7. **Execution sensitivity to goal position.** The Oracle Goal is translated with the planner bypassed. Each point pools 256 scenes at seeds 999 and 997 (512 episodes), with scenes and directions paired across magnitudes. Markers show measured success, error bars show pointwise 95% Wilson intervals, and lines guide the eye. PegInsertionSide uses 10 mm clearance.

D. Position Sensitivity

We bypass the planner and translate the Oracle Goal along directions sampled isotropically in the world frame, including vertical offsets. Offsets stay fixed per episode, with scenes and directions paired across magnitudes. Only each environment’s initial episode is scored. The two tasks span 8,192 rollouts over eight settings. PegInsertionSide uses widened 10 mm clearance. Error bars are pointwise 95% Wilson intervals, not intervals for paired effects or policy training variability.

Figure 7 shows success declining with larger offsets on both tasks, with a sharper StackCube drop from 30 to 50 mm. Each curve retains its measured zero-offset baseline rather than the pipeline Oracle reference. Goal distributions measure errors delivered by a pipeline, while this intervention measures the fixed executor’s response to imposed errors. These analyses separate planner-delivered goal error from controller sensitivity, while task success reflects their combined effect.

TABLE III
PLANNING COST ON AN RTX 5090 AFTER MODEL WARMUP USING THE SHARED TIMING AUDIT. TESSERACT REPORTS MEAN $\pm$ STANDARD DEVIATION OVER 18 CLIPS.

Planner	Parameters	Memory (GB)	Time (s)	Output
Cosmos 3 Edge	9.13B	19.33	35.63	81 RGB frames
Tesseract	5.7B	16.22	292.0 $\pm$ 4.9	49 RGB-DN frames
Track2Act	469M	1.91	3.80	400 2D tracks
Structured-4D	1.1B	5.71	4.45	4D point cloud
Ours	677.6M	5.60	1.27	3D traces + goal

TABLE IV
REAL ROBOT SUCCESS ON A FRANKA ARM WITH AN RTX 3090.

Task	Setting	Successes / Trials	Rate (%)
StackCube	nominal	22/30	73.33
PickPlate	unseen target	9/12	75.00
StackCube + distractors	intervention	8/12	66.67

The intervention directly tests whether the accuracy of the task variable exposed to control matters during execution.

E. Computational Cost and Real Robot Deployment

Table III reports the shared timing audit separately from training-evaluation timing. The readout adds less than 0.5% to planner parameters. The executor’s approximately 1.4 ms inference fits inside the 20 Hz loop, while planning runs only once per episode.

Real Robot Deployment. Franka trials with an RTX 3090 cover nominal StackCube, distractors, and PickPlate (Table IV, Fig. 8). Nominal StackCube reaches 73.33% success, StackCube with distractors reaches 66.67%, and PickPlate reaches 75.00%. These trials extend evaluation beyond nominal stacking. PickPlate is unseen during policy training but has a larger valid target region, providing evidence of target transfer rather than greater manipulation difficulty.

V. LIMITATIONS

A single terminal pose does not explicitly encode trajectory constraints, contact sequences, or articulated and deformable-object states. Planning occurs once per episode, so pose feedback can correct execution but cannot revise an invalidated goal. The system also depends on reliable tracking, depth, and pose estimates, while new goal semantics may require additional task-relevant supervision. Our results support task-aligned readout rather than the necessity of future prediction, since direct goal prediction may suffice for visually observable objectives.

VI. CONCLUSION

We presented Entity-Level Goal Readout, a learned prediction-to-execution interface that makes terminal task variables explicit learning targets through object-centric pose prediction and depth-grounded translation. The reported pipelines improve terminal goal quality and mean closed-loop success, while controlled perturbations show that positional goal error matters to a fixed executor. Real robot trials extend the interface to physical manipulation. The central

(a) StackCube · nominal



(b) StackCube · distractor intervention



(c) PickPlate · unseen target



Fig. 8. **Real robot deployment.** Rollouts show (a) nominal StackCube, (b) StackCube with distractors, and (c) PickPlate, where the cube is placed in the orange plate. PickPlate is unseen during policy training. Frames progress from left to right.

finding is that rich future predictions do not by themselves expose the task variables required by control. These variables must be explicitly learned and delivered through a reliable prediction-to-execution interface. Our system realizes this principle with terminal pose goals and online feedback. Richer goal representations, goal revision during execution, and uncertainty-aware perception remain directions for broader deployment.

ACKNOWLEDGMENT

Claude (Anthropic) and ChatGPT (OpenAI) assisted with manuscript text and figure layouts. The authors produced or verified scientific claims, experimental design, data collection, analysis, and results, assuming full responsibility.

REFERENCES

- [1] NVIDIA, “Cosmos 3 model reference,” Model documentation, 2026, accessed September 14, 2026. [Online]. Available: https://docs.nvidia.com/cosmos/latest/cosmos3/model_reference.html
- [2] Google DeepMind, “Veo 3.1,” Model documentation, 2025. [Online]. Available: <https://deepmind.google/models/veo/>
- [3] H. Zhen *et al.*, “Learning 4D embodied world models,” in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2025, pp. 5337–5347.
- [4] K. Dharmarajan, W. Huang, J. Wu, L. Fei-Fei, and R. Zhang, “Dream2Flow: Bridging video generation and open-world manipulation with 3D object flow,” *arXiv preprint arXiv:2512.24766*, 2025.
- [5] C. Wen *et al.*, “Any-point trajectory modeling for policy learning,” in *Proc. Robotics: Science and Systems (RSS)*, 2024.
- [6] H. Bharadhwaj, R. Mottaghi, A. Gupta, and S. Tulsiani, “Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation,” in *Proc. European Conf. on Computer Vision (ECCV)*, 2024.
- [7] S. Lee *et al.*, “TraceGen: World modeling in 3D trace space enables learning from cross-embodiment videos,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [8] S. Lee *et al.*, “ μ_0 : A scalable 3D interaction-trace world model,” *arXiv preprint arXiv:2606.13769*, 2026.
- [9] W. Huang *et al.*, “PointWorld: Scaling 3D world models for in-the-wild robotic manipulation,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [10] Z. Li, P. Wu, X. Han, R. Cai, and Y. Du, “Structured 4D latent predictive model for robot planning,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2026.
- [11] W. Kabsch, “A solution for the best rotation to relate two sets of vectors,” *Acta Crystallographica Section A*, vol. 32, no. 5, pp. 922–923, 1976.
- [12] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [13] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” *Int. J. Robotics Research*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [14] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, “3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations,” in *Proc. Robotics: Science and Systems (RSS)*, 2024.
- [15] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- [16] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, “Learning by cheating,” in *Proc. Conf. on Robot Learning (CoRL)*, 2020.
- [17] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, “Learning quadrupedal locomotion over challenging terrain,” *Science Robotics*, vol. 5, no. 47, p. eabc5986, 2020.
- [18] O. Siméoni *et al.*, “DINOv3,” *arXiv preprint arXiv:2508.10104*, 2025.
- [19] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2023.
- [20] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [21] Z. Yang *et al.*, “CogVideoX: Text-to-video diffusion models with an expert transformer,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.

- [22] N. Karaev, Y. Makarov, J. Wang, N. Neverova, A. Vedaldi, and C. Rupprecht, “CoTracker3: Simpler and better point tracking by pseudo-labelling real videos,” in *Proc. IEEE/CVF Int. Conf. on Computer Vision (ICCV)*, 2025.
- [23] S. Tao, F. Xiang, A. Shukla *et al.*, “Demonstrating GPU parallelized robot simulation and rendering for generalizable embodied AI with ManiSkill3,” in *Proc. Robotics: Science and Systems (RSS)*, 2025.
- [24] S. Lin *et al.*, “RoboFlow4D: A lightweight flow world model toward real-time flow-guided robotic manipulation,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2026.